EXPLORING NUMPY & PANDAS

NumPy is a general-purpose array-processing python package. It provides a high-performance multidimensional array object, and tools for working with these arrays. It is the fundamental package for scientific computing with Python.

- It is a table of elements (usually numbers), all the same type, indexed by a tuple of positive integers.
- In NumPy dimensions are called *axes*. The number of axes is *rank*.
- NumPy's array class is called **ndarray**. It is also known by the alias **array**.

Example:

[[1, 2, 3],
[4, 2, 5]]

Here,

- rank = 2 (as it is 2-dimensional or it has 2 axes)
- first dimension(axis) length = 2, second dimension has length = 3
- overall shape can be expressed as: (2, 3)

1.Numpy Arrays

1.Program to demonstrate array creation

```
import numpy as np
arr = np.array( [[ 1, 2, 3],
                 [ 4, 2, 5]] )
print("Array is of type: ", type(arr))
print("No. of dimensions: ", arr.ndim)
print("Shape of array: ", arr.shape)
print("Size of array: ", arr.size)
print("Array stores elements of type: ", arr.dtype)
```

output:

```
Array is of type:
No. of dimensions:  2
Shape of array:  (2, 3)
Size of array:  6
Array stores elements of type:  int64
```

2.Program to demonstrate array

```
import numpy as np

# Creating array from list with type float
a = np.array([[1, 2, 4], [5, 8, 7]], dtype = 'float')
print ("Array created using passed list:\n", a)

# Creating array from tuple
b = np.array((1 , 3, 2))
print ("\nArray created using passed tuple:\n", b)

# Creating a 3X4 array with all zeros
c = np.zeros((3, 4))
print ("\nAn array initialized with all zeros:\n", c)

# Create a constant value array of complex type
d = np.full((3, 3), 6, dtype = 'complex')
print ("\nAn array initialized with all 6s." ,"Array type is complex:\n", d)

# Create an array with random values
e = np.random.random((2, 2))
print ("\nA random array:\n", e)

# Create a sequence of integers
# from 0 to 30 with steps of 5
f = np.arange(0, 30, 5)
print ("\nA sequential array with steps of 5:\n", f)

# Create a sequence of 10 values in range 0 to 5
g = np.linspace(0, 5, 10)
print ("\nA sequential array with 10 values between"
                                        "0 and 5:\n", g)

# Reshaping 3X4 array to 2X2X3 array
arr = np.array([[1, 2, 3, 4],
                [5, 2, 4, 2],
                [1, 2, 0, 1]])
newarr = arr.reshape(2, 2, 3)
```

```
print ("\nOriginal array:\n", arr)
print ("Reshaped array:\n", newarr)

# Flatten array
arr = np.array([[1, 2, 3], [4, 5, 6]])
flarr = arr.flatten()

print ("\nOriginal array:\n", arr)
print ("Fattened array:\n", flarr)
```

3.*Program to demonstrate basic array operations*

```
import numpy as np
a = np.array([1, 2, 5, 3])

# add 1 to every element
print ("Adding 1 to every element:", a+1)

# subtract 3 from each element
print ("Subtracting 3 from each element:", a-3)

# multiply each element by 10
print ("Multiplying each element by 10:", a*10)

# square each element
print ("Squaring each element:", a**2)

# modify existing array
a *= 2
print ("Doubled each element of original array:", a)

# transpose of array
a = np.array([[1, 2, 3], [3, 4, 5], [9, 6, 0]])

print ("\nOriginal array:\n", a)
print ("Transpose of array:\n", a.T)
```

output:

```
Adding 1 to every element: [2 3 6 4]
Subtracting 3 from each element: [-2 -1  2  0]
Multiplying each element by 10: [10 20 50 30]
Squaring each element: [ 1  4 25  9]
Doubled each element of original array: [ 2  4 10  6]

Original array:
 [[1 2 3]
 [3 4 5]
 [9 6 0]]
Transpose of array:
 [[1 3 9]
 [2 4 6]
 [3 5 0]]
```

2.Array Aggregation Functions

Unary operators: Many unary operations are provided as a method of **ndarray** class. This includes sum, min, max, etc. These functions can also be applied row-wise or column-wise by setting an axis parameter.

4.***Program to demonstrate array aggregate functions:***

```
import numpy as np
arr = np.array([[1, 5, 6],
                [4, 7, 2],
                [3, 1, 9]])

# maximum element of array
print ("Largest element is:", arr.max())
print ("Row-wise maximum elements:",arr.max(axis = 1))

# minimum element of array
print ("Column-wise minimum elements:",arr.min(axis = 0))

# sum of array elements
print ("Sum of all array elements:",arr.sum())

# cumulative sum along each row
print ("Cumulative sum along each row:\n",arr.cumsum(axis = 1))
```

output:
```
Largest element is: 9
Row-wise maximum elements: [6 7 9]
Column-wise minimum elements: [1 1 2]
Sum of all array elements: 38
Cumulative sum along each row:
[[ 1  6 12]
 [ 4 11 13]
 [ 3  4 13]]
```

5. *Array aggregate functions:*

```
import numpy as np
arr1 = np.array([10, 20, 30, 40, 50])
print(arr1)

arr2 = np.array([[0, 10, 20], [30, 40, 50], [60, 70, 80]])
print(arr2)

arr3 = np.array([[14, 6, 9, -12, 19, 72],[-9, 8, 22, 0, 99, -11]])
print(arr3)

print('the sum of array 1, 2, 3 respectively:')
print(arr1.sum())
print(arr2.sum())
print(arr3.sum())

print('the average of array 1, 2, 3 respectively:')
print(np.average(arr1))
print(np.average(arr2))
print(np.average(arr3))

print('the min of array1 and max of array 2:')
print(arr1.min())
print(arr2.max())
```

Binary operators: These operations apply on array elementwise and a new array is created. You can use all basic arithmetic operators like +, -, /, , *etc. In case of* +=, -=, = operators, the existing array is modified.

```
import numpy as np
a = np.array([[1, 2],
              [3, 4]])

b = np.array([[4, 3],
              [2, 1]])

print ("Array sum:\n", a + b)
print ("Array multiplication:\n", a*b)
print ("Matrix multiplication:\n", a.dot(b))
```

3.Use Map, Filter, Reduce and Lambda Functions with NumPy

1.lambda function:

- Lambda function is also known as anonymous (without a name) function that directly accepts the number of arguments and a condition or operation to perform with that argument which is colon-separated and returns the result.

- To perform a small task while coding on a large codebase or a small task inside a function, we use the lambda function in a normal process

- higher-order functions are the functions that need one more function in them to accomplish their task, or when one function is returning any another function, then we use Lambda functions.

Examples of lambda function

```
x= lambda a:a+ 10
print(x(5))
```

```
x= lambda a,b:a*b
print(x(5, 6))
```

```
sum = lambda x, y : x + y
sum(3,4)
```

Example of lambda function using if-else

Max = lambda a, b : a if(a > b) else b
print(Max(1, 2))

2. Map Function:

Syntax: map(fun, iter)

- The map function is a function that accepts two parameters. The first is a function, and the second is any sequence data type that is iterable.
- The Map function work is to apply some operation defined in each element of the iterator object.
- Suppose we want to change the array elements into a square array means we want a square of each component of a variety to map a square function with an array that will produce the required result.

```
arr = [2,4,6,8]
arr = list(map(lambda x: x*x, arr))
print(arr)
```

```
a = [1, 2, 3, 4]
b = [17, 12, 11, 10]
c = [-1, -4, 5, 9]
list(map(lambda x, y, z : x+y+z, a, b, c))
```

Python program to demonstrate working of map.
Return double of n

def addition(n):
    return n + n

We double all numbers using map()

numbers = (1, 2, 3, 4)
result = map(addition, numbers)
print(list(result))

Double all numbers using map and lambda

numbers = (1, 2, 3, 4)
result = map(lambda x: x + x, numbers)
print(list(result))

3. Filter Function:

- Filter function filters out the data based on a particularly given condition.
- Map function operates on each element, while filter function only outputs elements that satisfy specific requirements.

Suppose we have a list of fruits, and our task is to output only those names which have the character "g" in their name.

```
fruits = ['mango', 'apple', 'orange', 'cherry', 'grapes']
print(list(filter(lambda fruit: 'g' in fruit, fruits)))
```

fibonacci = [0,1,1,2,3,5,8,13,21,34,55]
odd_numbers = list(filter(lambda x: x % 2, fibonacci))
print(odd_numbers)

4. Reduce Function

- import it from the functional tools' module of Python.
- Reduce returns a single output value from a sequence data structure because it reduces the elements by applying a given function.

Suppose we have a list of integers and find the sum of all the aspects. Then instead of using for loop, we can use reduce function.

```
from functools import reduce
lst = [2,4,6,8,10]
print(reduce(lambda x, y: x+y, lst))  #Ans-30
```

```
lst = [2,4,6,8]
#find largest element
print(reduce(lambda x, y: x if x>y else y, lst))
#find smallest element
print (reduce(lambda x, y: x if x<y else y, lst))
```

4.Pd-Series

- Pandas Series is a one-dimensional array capable of holding data of any type (integer, string, float, python objects, etc.). The axis labels are collectively called *index*.
- Pandas Series can be created from the lists, dictionary, and from a scalar value etc

Creating series using pd.Series()

```
import pandas as pd
import numpy as np
info = np.array(['P','a','n','d','a','s'])
a = pd.Series(info)
print(a)
```

output

```
0  P
1  a
2  n
3  d
4  a
5  s
```

Creating series from a scalar

```
#import pandas library
import pandas as pd
import numpy as np
x = pd.Series(4, index=[0, 1, 2, 3])
print (x)
```

pd.Series value_counts() function

```
import pandas as pd
import numpy as np
index = pd.Index([2, 1, 1, np.nan, 3])
index.value_counts()
```

output

```
1.0    2
3.0    1
2.0    1
dtype: int64
```

Get the items which are not common of two Pandas series

```
# import the modules

import pandas as pd
import numpy as np
ser1 = pd.Series([1, 2, 3, 4, 5])
ser2 = pd.Series([3, 4, 5, 6, 7])
union = pd.Series(np.union1d(ser1, ser2))
intersect = pd.Series(np.intersect1d(ser1, ser2))
notcommonseries = union[~union.isin(intersect)]
print(notcommonseries)
```

5.Pandas DataFrame

Data Frame is **a 2-dimensional labelled data structure with columns of potentially different types**. Dataframe is like a spreadsheet or SQL table, or a dict of Series objects. It is generally the most used panda's object.

```
# Import module
import pandas as pd
# Creating our dataset
df = pd.DataFrame([ [9, 4, 8, 9],
                    [8, 10, 7, 6],
                    [7, 6, 8, 5]],
                        columns=['Maths', 'English',
                                 'Science', 'History'])
print(df)
```

```
import pandas as pd

# intialise data of lists.
data = {'Name':['Tom', 'nick', 'krish', 'jack'],
        'Age':[20, 21, 19, 18]}

# Create DataFrame
df = pd.DataFrame(data)

# Print the output.
print(df)
```

Output:

	Name	Age
0	Tom	20
1	nick	21
2	krish	19
3	jack	18

Pandas aggregation and grouping

```
import pandas as pd
df = pd.DataFrame([[9, 4, 8, 9],
                   [8, 10, 7, 6],
                   [7, 6, 8, 5]],
                   columns=['Maths', 'English',
                            'Science', 'History'])

print(df)
print(df.sum())
print(df.describe())
print(df.agg(['sum', 'min', 'max']))
a = df.groupby('Maths')
a.first()
```

Solve aggregate using Dataset

Pandas Pivot () and melt() functions

Pandas melt() function is used to change the DataFrame format from wide to long. It's used to create a specific format of the DataFrame object where one or more columns work as identifiers. All the remaining columns are treated as values and unpivoted to the row axis and only two columns - variable and value.

```
import pandas as pd
calories = {"day1": 420, "day2": 380, "day3": 390}
myvar = pd.Series(calories)
print(myvar)
```

```
Df=pd.DataFrame({'Newyork':
[25], 'Paris': [27], 'London':
[30]})

                                    df
Print ( df.melt() )

df_larger=pd.DataFrame({
                                    'New york': [25, 27, 23, 25, 29],
                                    'Paris': [27, 22, 24, 26, 28],
                                    'London': [30, 31, 33, 29, 25]
                                })

Print(df_larger.melt())
```

Pivot table

Syntax: pandas.pivot_table(data, values=None, index=None, columns=None, aggfunc='mean', fill_value=None, margins=False, dropna=True, margins_name='All')

```
# Create a simple dataframe
import pandas as pd
import numpy as np

df = pd.DataFrame({'A': ['John', 'Boby', 'Mina', 'Peter', 'Nicky'],
        'B': ['Masters', 'Graduate', 'Graduate', 'Masters', 'Graduate'],
        'C': [27, 23, 21, 23, 24]})
```

Print(df)

	A	B	C
0	John	Masters	27
1	Boby	Graduate	23
2	Mina	Graduate	21
3	Peter	Masters	23
4	Nicky	Graduate	24

table = pd.pivot_table(df, index =['A', 'B'])

print(table)

Out[2]:

		C
A	**B**	
Boby	**Graduate**	23
John	**Masters**	27
Mina	**Graduate**	21
Nicky	**Graduate**	24
Peter	**Masters**	23

table = pd.pivot_table(df, values ='A', index =['B', 'C'],

columns =['B'], aggfunc = np.sum)

print(table)

Out[3]:

	B	Graduate	Masters
B	**C**		
Graduate	**21**	Mina	NaN
	23	Boby	NaN
	24	Nicky	NaN
Masters	**23**	NaN	Peter
	27	NaN	John

Create DataFrame and find pivot table

Diploma

	First Name	Last Name	Type	Department	YoE	Salary
0	Aryan	Singh	Full-time Employee	Administration	2	20000
1	Rohan	Agarwal	Intern	Technical	3	5000
2	Riya	Shah	Full-time Employee	Administration	5	10000
3	Yash	Bhatia	Part-time Employee	Technical	7	10000
4	Siddhant	Khanna	Full-time Employee	Management	6	20000

1. Make a pivot table which shows average salary of each type of employee for each department.
2. Make a pivot table which shows the sum and mean of the salaries of each type of employee and the number of employees of each type.

Working with Missing Values:

- Missing Data can occur when no information is provided for one or more items or for a whole unit. Missing Data is a very big problem in a real-life scenario. Missing Data can also refer to as NA (Not Available) values in pandas. (NAN- not a number)
- In DataFrame sometimes many datasets simply arrive with missing data, either because it exists and was not collected or it never existed.

Pandas treat None and NaN as essentially interchangeable for indicating missing or null values. To facilitate this convention, there are several useful functions for detecting, removing, and replacing null values in Pandas DataFrame:

- **isnull()**
- **notnull()**
- **dropna()**
- **fillna()**
- **replace ()**
- **interpolate()**

Program to demonstrate isnull()

```
import numpy as np
dict = {'First Score':[100, 90, np.nan, 95],
                'Second Score': [30, 45, 56, np.nan],
                'Third Score':[np.nan, 40, 80, 98]}

df = pd.DataFrame(dict)
print(df.isnull())
print(df.notnull())
print(df.fillna(0))
print(df.fillna(method='pad'))
print(df.fillna(method='bfill))
```

output: observe the output

import Employee Csv file and perform fill the missing values operation:

```
import pandas as pd
data = pd.read_csv("employees.csv")

# filling a null values using fillna()
data["Gender"].fillna("No Gender", inplace = True)

print(data)
```

Replace function

```
import pandas as pd
data = pd.read_csv("employees.csv")

# will replace Nan value in dataframe with value -99
data.replace(to_replace = np.nan, value = -99)
```

To drop rows containing missing values

```
import pandas as pd
import numpy as np

# dictionary of lists
dict = {'First Score':[100, 90, np.nan, 95],
                'Second Score': [30, np.nan, 45, 56],
                'Third Score':[52, 40, 80, 98],
                'Fourth Score':[np.nan, np.nan, np.nan, 65]}
df = pd.DataFrame(dict)

df.dropna()                    # drop rows containing missing values

df.dropna(how = 'all')         #drop rows only when it contains missing values in all the column
```

Data Visualization

- In today's world, a lot of data is being generated daily. And sometimes there is a need to analyse this data for certain trends, patterns may become difficult if the data is in its raw format.
- To overcome this data visualization comes into play.
- Data visualization provides a good, organized pictorial representation of the data which makes it easier to understand, observe, analyse. In this tutorial, we will discuss how to visualize data using Python.
- Python provides various libraries that come with different features for visualizing data.

1. Matplotlib
2. Seaborn
3. Plotly

Matplotlib

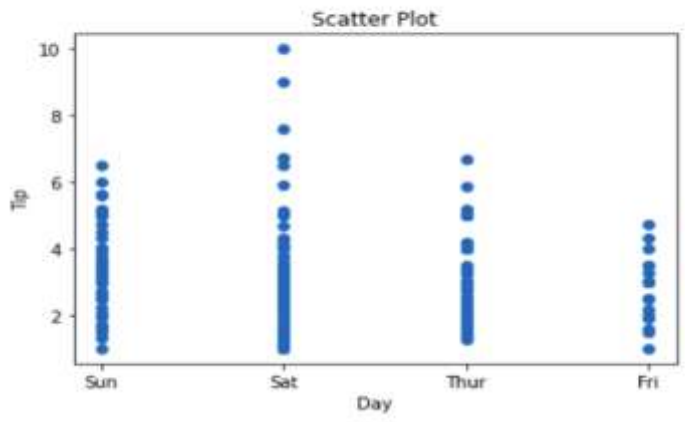
- Matplotlib is an easy-to-use, low-level data visualization library that is built on NumPy arrays. It consists of various plots like scatter plot, line plot, histogram, etc. Matplotlib provides a lot of flexibility.

- Tips database is the record of the tip given by the customers in a restaurant for two and a half months in the early 1990s. It contains 6 columns such as total_bill, tip, sex, smoker, day, time, size.

1.Scatter plots are used to observe relationships between variables and uses dots to represent the relationship between them. The **scatter()** method in the matplotlib library is used to draw a scatter plot.

```
import pandas as pd
import matplotlib.pyplot as plt
data = pd.read_csv("tips.csv")
plt.scatter(data['day'], data['tip'])
plt.title("Scatter Plot")
plt.xlabel('Day')
plt.ylabel('Tip')
plt.show()
```

Output:



Color can be changed by using c and s parameters

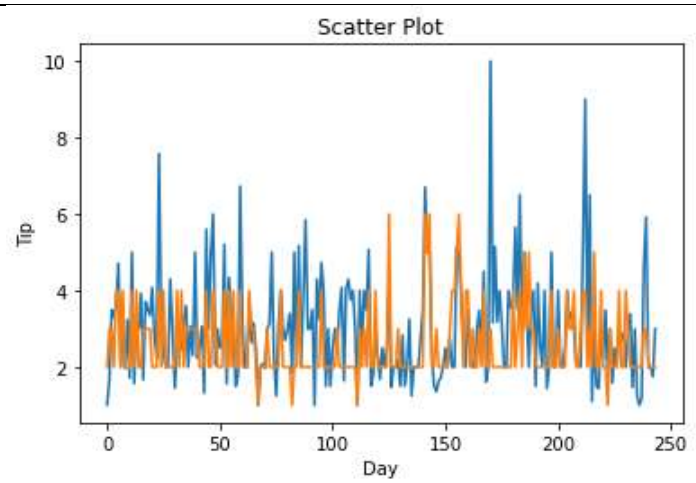
Or plt.scatter(data['day'],data['tip'], ??)

plt.colourbar() function

2.line chart

Line Chart is used to represent a relationship between two data X and Y on a different axis. It is plotted using the **plot()** function.

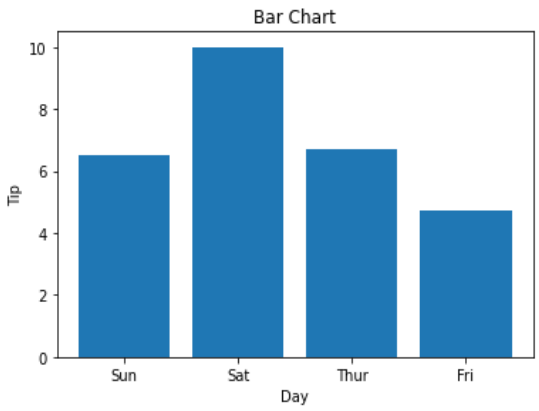
```
import pandas as pd
import matplotlib.pyplot as plt
data = pd.read_csv("tips.csv")
plt.plot(data['tip'])
plt.plot(data['size'])
plt.title("Scatter Plot")
# Setting the X and Y labels
plt.xlabel('Day')
plt.ylabel('Tip')
plt.show()
```



3.Bar Chart

A bar plot or bar chart is a graph that represents the category of data with rectangular bars with lengths and heights that is proportional to the values which they represent. It can be created using the **bar ()** method.

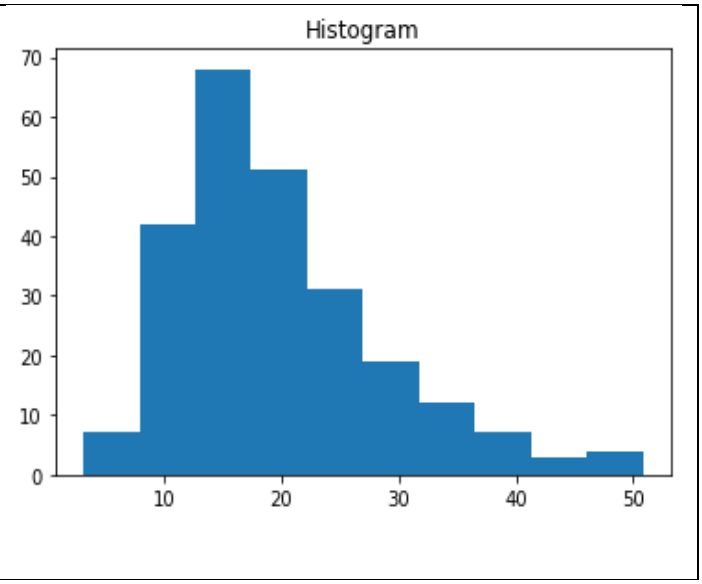
```
import pandas as pd

import matplotlib. pyplot as plt

data = pd.read_csv("tips.csv")

plt.bar(data['day'], data['tip'])

plt.title("Bar Chart")

plt.xlabel('Day')

plt.ylabel('Tip')

plt.show()
```



4. Histogram

- A histogram is basically used to represent data in the form of some groups.
- It is a type of bar plot where the X-axis represents the bin ranges while the Y-axis gives information about frequency.
- The **hist()** function is used to compute and create a histogram. In histogram, if we pass categorical data then it will automatically compute the frequency of that data i.e. how often each value occurred.

```
import pandas as pd

import matplotlib.pyplot as plt

data = pd.read_csv("tips.csv")

plt.hist(data['total_bill'])

plt.title("Histogram")

plt.show()
```



Titanic Dataset –

Case 1:

It is one of the most popular datasets used for understanding machine learning basics. It contains information of all the passengers aboard the RMS Titanic, which unfortunately was shipwrecked. This dataset can be used to predict whether a given passenger survived or not.

1. PassengerId: Unique Id of a passenger
2. Survived: If the passenger survived(0-No, 1-Yes)
3. Pclass: Passenger Class (1 = 1st, 2 = 2nd, 3 = 3rd)
4. Name: Name of the passenger
5. Sex: Male/Female
6. Age: Passenger age in years
7. SibSp: No of siblings/spouses aboard
8. Parch: No of parents/children aboard
9. Ticket: Ticket Number
10. Fare: Passenger Fare
11. Cabin: Cabin number
12. Embarked: Port of Embarkation (C = Cherbourg; Q = Queenstown; S = Southampton)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
data=pd.read_csv('C:/Users/Shilpa/Desktop/dataset/titanic/train.csv')
print(data.shape)
print(data.info())
print(data.isnull().sum())
print(len(data))
print(data.head(10))
print(data['Survived'].value_counts().plot(kind='bar'))
print(data[data['Sex']=='female']['Survived'].value_counts().plot(kind='bar'))
print(data[data['Pclass']==3]['Survived'].value_counts().plot(kind='bar'))
plt.scatter(data['Fare'],data['Pclass'])
print(data[data['Sex']=='female']['Survived'].value_counts().plot(kind='pie'))
import seaborn as sns
sns.barplot(x='Pclass', y='Survived', data=data)
plt.scatter(x=data['Age'],y=data['Fare'])
plt.hist(data['Fare'])
```

case 2:

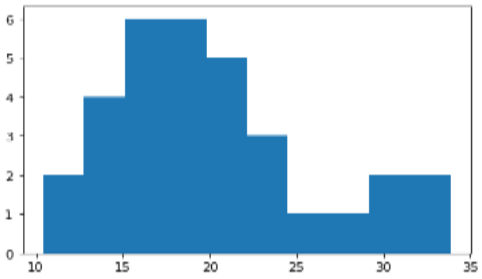
Data from an online platform has been collected. This data contains fuel consumption and 11 aspects of automobile design and performance for 32 automobiles. Variable description is given below. Create the following plots to visualize/summarize the data and customize appropriately

1. **histogram to check the frequency distribution of the variable 'mpg' (Miles per gallon) and note down the interval having the highest frequency.**
2. **scatter plot to determine the relation between weight of the car and mpg**
3. **bar plot to check the frequency distribution of transmission type of cars.**
4. **Box and Whisker plot of mpg and interpret the five number summary.**
5. **Create a git repository and push source code to repo.**

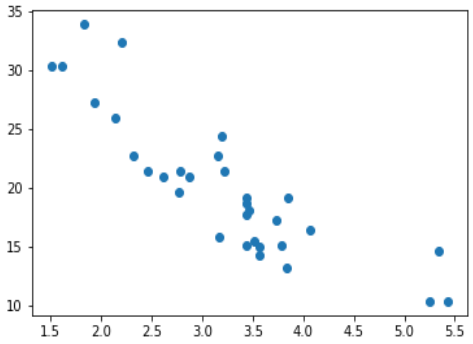
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df=pd.read_csv('C:/Users/SPTINT-06/Desktop/mtcars.csv')
df.head(10)
```

```
plt.hist(x=df['mpg'])
```

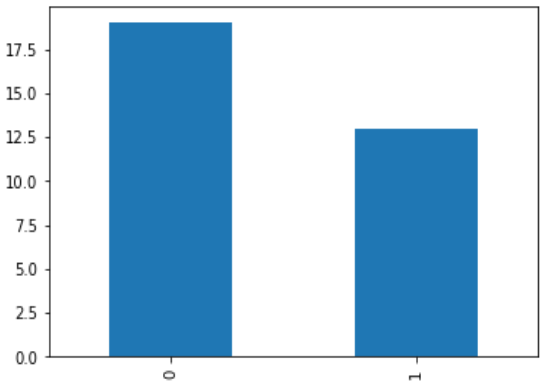
```
(array([2., 4., 6., 6., 5., 3., 1., 1., 2., 2.]),
 array([10.4 , 12.75, 15.1 , 17.45, 19.8 , 22.15, 24.5 , 26.85, 29.2 ,
        31.55, 33.9 ]),
```



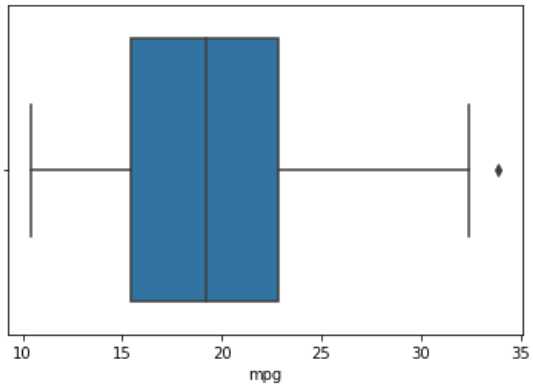

```
plt.scatter(x='wt',y='mpg',data=df)
```




```
df['am'].value_counts().plot(kind='bar')
```



```
sns.boxplot(df['mpg'])
```




```
df['mpg'].min()
10.4

df['mpg'].max()
33.9

df['mpg'].quantile([.1, .25, .5, .75])

0.10    14.340
0.25    15.425
0.50    19.200
0.75    22.800
Name: mpg, dtype: float64
```

Five number summary.

Case 3:

2c. Ramesh decides to walk 10000 steps every day to combat the effect that lockdown has had on his body's agility, mobility, flexibility and strength. Consider the following data from fitness tracker over a period of 10 days

10M

	day	steps
0	1	4335
1	2	9552
2	3	7332
3	4	4504
4	5	5335
5	6	7552
6	7	8332
7	8	6504
8	9	8965
9	10	7689

a) Perform an appropriate operation to add 1000 steps to all the observations
b) Find out the days on which he walked more than 7000 steps

1a)

```
import pandas as pd
steps_tracker= {'day': [1,2,3,4,5,6,7,8,9,10], 'steps': [4335,9552,7332,4504,5335,7552,8332,6504,8965,7689]}
df = pd.DataFrame(steps_tracker)

print("\n  Steps after adding 1000 steps to all observations")
steps_tracker['steps']=df['steps']+1000
print(steps_tracker['steps'])
```

1b)

```
import pandas as pd
steps_tracker= {'day': [1,2,3,4,5,6,7,8,9,10], 'steps': [4335,9552,7332,4504,5335,7552,8332,6504,8965,7689]}
df = pd.DataFrame(steps_tracker)

print("\n Days on which Ramesh walks more than 7000 steps")
days_more_than_7k = df.loc[df['steps'] > 7000]
print(days_more_than_7k)
```

Case 4:

2.c) Create the DataFrame for the table below:

	First_name	Branch	Location_type	semester	marks
0	raju	computers	home	5	450
1	ankitha	electronics	PG	3	591
2	purvi	computers	home	5	482
3	swetha	electronics	hostel	4	378
4	pallavi	mechanical	home	5	538

i) Make a pivot table to show Average marks of each location_type of students of each branch.

ii) Make a pivot table to show sum and mean of marks of each location_type of students and numbers of students of each type

```
Import pandas as pd
Import numpy as np
Data=pd.DataFrame({'firstname':['raju','ankitha','purni','swetha','pallavi',],'branch:['compute
r','electronics',
'computer','mechanical'],'loactiontype':['home','pg','home','hostel','home'],''semester':[5,3,5
,4,5],''marks':[450,532,482,378,538]})

a=pd.Pivot_table(df, index=['location.type','branch'],value=['marks'],aggfunc=np.mean)
Print(a)

b=pd.Pivot_table(df,
index=['location.type','branch'],value=['marks'],aggfunc=[np.mean,np.sum,pd.series.unique])
Print(b)
```

Case 5: The conventional method of taking attendance is done manually by the teacher or the administrator by calling out student's names, which requires considerable amount of time and efforts. As the number of students is increasing day by day, it is a challenging task for universities or colleges to monitor and maintain the record of the students. What are the implications concerned with conventional method and suggest the solution using machine learning to get better of the same.

The conventional method of taking attendance has lot of implications:

1. It takes more time and effort.
2. The physical recording of attendance may get damaged or lost.
3. It requires huge amount of time to retrieve the data if it is done manually.
4. May be recording errors during taking attendance.
5. Forge may happen.

To these scenarios Al enabled solution can help better in marking attendance and avoiding the above-mentioned implications. The universities and college may go with AI enabled bio-metric based attendance system, like face detection, or RFID card reader. With this system it helps to make attendance accurately without any error and also helps in management of data more precisely and accurately.